ORGNZ-01: Introduction to Organizing Data in Grail

> **Series:** ORGNZ | **Notebook:** 1 of 10 | **Created:** January 2026 | **Last Updated:** 02/09/2026

Overview

Dynatrace Grail organizes data in **buckets**, **tables**, and **views** to ensure efficient storage, flexible access, and scalable querying. Understanding how to organize your data is fundamental to achieving optimal query performance, compliance requirements, and access control.

Prerequisites

Requirement	Details

Table of Contents

----------|----------|
Dynatrace Environment	SaaS environment with Grail enabled
Permissions	`storage:buckets:read` for viewing data organization
Knowledge	Basic familiarity with Dynatrace and DQL

Learning Objectives

By the end of this notebook, you will:
- Understand why data organization matters in Grail
- Know the three pillars of data organization: buckets, segments, and security
- Recognize when to use each organization mechanism
- Understand the Grail data model structure

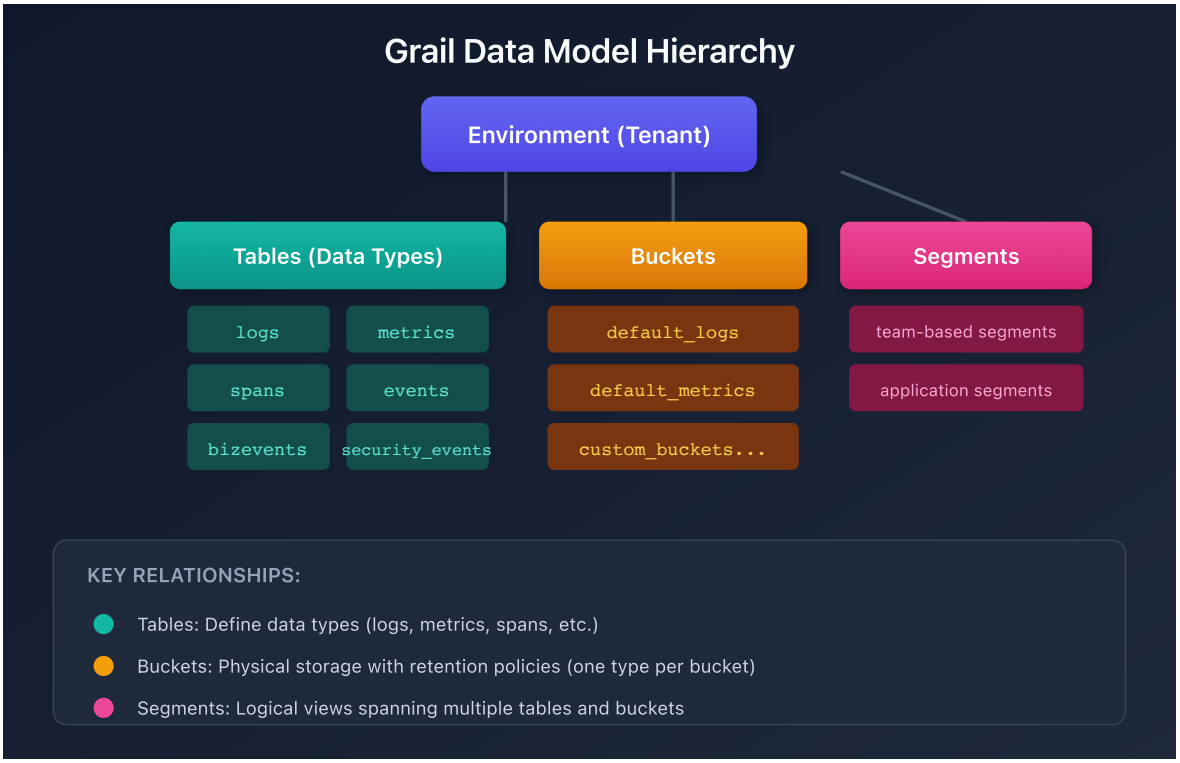
Why Organize Data?

Effective data organization in Grail enables:

Benefit	Description
Query Performance	Reduce query execution time by limiting data scope
Cost Control	Transparent GiB/day pricing with bucket-level attribution
Access Control	Fine-grained permissions from bucket to field level
Compliance	Meet retention and data residency requirements
Operational Clarity	Teams see only relevant data for faster troubleshooting

The Grail Data Model

Grail organizes data hierarchically with tables (data types), buckets (storage containers), and segments (logical views):



Three Pillars of Data Organization

1. Buckets (Physical Organization)

Buckets are logical storage containers where records are stored:

Aspect	Description
Purpose	Physical data partitioning and retention control
Scope	One data type per bucket (logs, metrics, etc.)
Retention	1 day to 10 years per bucket
Access	IAM policies can grant bucket-level access
Cost	Enable precise cost attribution by team/LOB

2. Segments (Logical Organization)

Segments provide dynamic, cross-signal data views:

Aspect	Description
Purpose	Real-time filtering without physical separation
Scope	Spans multiple data types and buckets
Flexibility	Rule-based, can use variables and relationships
Access	Governed by existing access controls
Cost	No direct storage cost impact

3. Security Context (Access Control)

Security context enables fine-grained permissions:

Aspect	Description
Purpose	Record-level access control
Mechanism	`dt.security_context` attribute on records
Flexibility	Custom values, hierarchical encoding
Enforcement	IAM policies filter at query time

When to Use Each Mechanism

Need	Solution	Rationale
Different retention periods	**Buckets**	Retention is set per bucket
Cost attribution by team	**Buckets**	Direct GiB/day billing visibility
Broad team isolation	**Buckets**	Simple bucket-level IAM policies
Dynamic data views	**Segments**	Filter across data types at query time
Application-centric views	**Segments**	Group related entities and signals
Record-level access control	**Security Context**	Fine-grained ABAC
Multi-team shared buckets	**Security Context**	Filter records within same bucket

Permission Levels in Grail

Grail supports permissions at multiple levels:

Level	Granularity	Example Use Case
Bucket	All records in a bucket	Team owns entire bucket
Table	All records of a data type	Access to all logs
Record	Individual records by attribute	By host group, namespace, security context
Field	Specific fields on records	Mask sensitive fields

> **Important**: Without permissions, users cannot query data from Grail. Permissions must be explicitly granted.

Exploring Your Data Organization

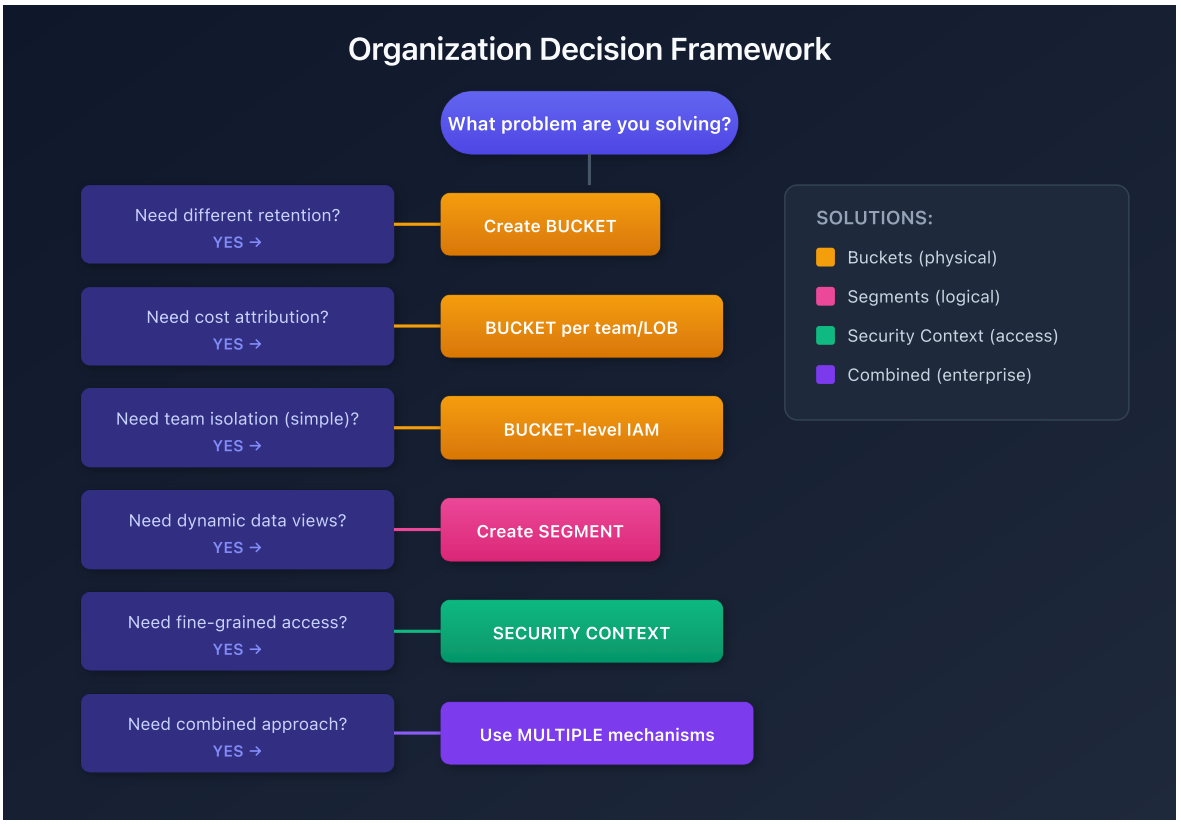
```
// List all buckets in your environment
fetch dt.system.buckets
| fields name, display_name, dt.system.table, retention_days,
estimated_uncompressed_bytes
| sort dt.system.table asc, name asc
```

```
// Summarize data volume by bucket
fetch logs, from:-1h
| summarize recordCount = count(), by:{dt.system.bucket}
| sort recordCount desc
```

```
// Check for records with security context
fetch logs, from:-1h
| filter isNotNull(dt.security_context)
| summarize count = count(), by:{dt.security_context}
| sort count desc
| limit 20
```

Organization Decision Framework

Use this framework to decide how to organize your data:



Series Overview

This series covers data organization in depth:

Notebook	Topic	Focus
ORGNZ-01	Introduction	Why and how to organize data
ORGNZ-02	Grail Buckets	Bucket fundamentals and limits
ORGNZ-03	Bucket Strategy	Naming, retention, design patterns
ORGNZ-04	Permissions Overview	Permission levels and policy management
ORGNZ-05	Bucket-Level Access	IAM policies for bucket access
ORGNZ-06	Security Context	Fine-grained access with dt.security_context
ORGNZ-07	Advanced Permissions	Record-level and field-based access
ORGNZ-08	Grail Segments	Dynamic data organization
ORGNZ-09	Enterprise Patterns	Combined approaches and best practices
ORGNZ-10	Advanced Segment Definitions	Filter syntax, enrichment, variables, troubleshooting

Next Steps

Continue with the ORGNZ series:
- **ORGNZ-02**: Understanding Grail Buckets

References

- Organize data
- Permissions in Grail
- Grail data model

This notebook was AI-generated from Dynatrace documentation and enterprise best practices. It is not officially supported by Dynatrace. Always verify information against official Dynatrace documentation.